How YouTube Was Able to Support 2.49 Billion Users With MySQL

#48: Break Into Vitess Architecture (4 minutes)

NEO KIM

MAY 31, 2024

Get my system design playbook for FREE on newsletter signup:

Subscribed

This post outlines Vitess architecture. If you want to learn more, find references at the bottom of the page.

- *Share this post & I'll send you some rewards for the referrals.*

Note: This post is based on my research and may differ from real-world implementation.

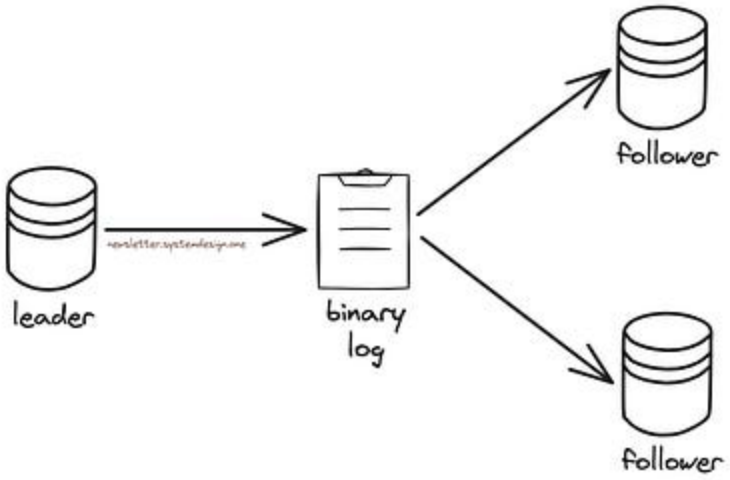
Once upon a time, 3 people working for PayPal decided to build a dating site.

Yet their business model failed.

So they pivoted to create a video-sharing site and called it YouTube.

They stored video titles, descriptions, and user data in MySQL.

As more users joined, they set up MySQL in leader-follower replication topology to scale.

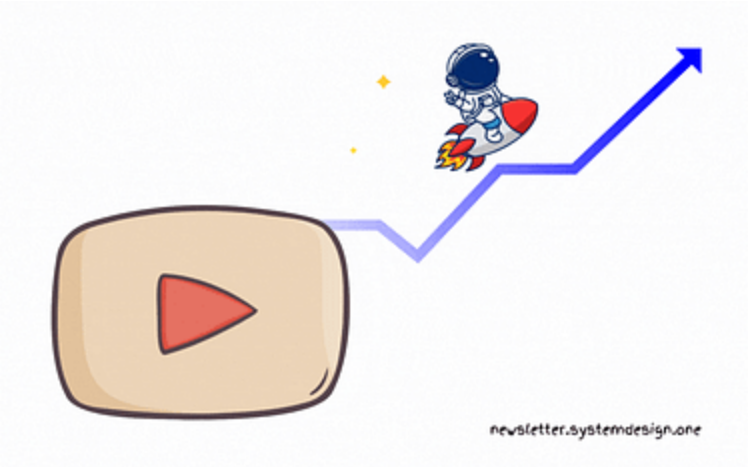


Leader-Follower Replication Topology in MySQL

But replication in MySQL is single-threaded.

So followers couldn't keep up with fresh data on extreme write operations to the leader.

Yet their growth rate was explosive.



And hit a whopping billion users to become the second most visited site in the world.

So they scaled out by adding a cache and preloaded all the events from the MySQL binary log. That means the replication becomes memory-bound and faster.

Although it temporarily solved their scalability issue, there were new problems.

Here are some of them:

1. Sharding:

MySQL must be partitioned to handle storage needs.

But transactions and joins become difficult after sharding.

So application logic should handle it.

This means application logic should find what shards to query.

And that increases the chance of downtime.

2. Performance:

The leader-follower replication topology causes stale data to be read from followers.

So application logic must route the reads to the leader if fresh data is necessary.

And this needs extra logic implementation.

3. Protection:

There's a risk of some queries taking too long to return data.

Also too many MySQL connections at once can be problematic.

And might take down the database.

Vitess MySQL

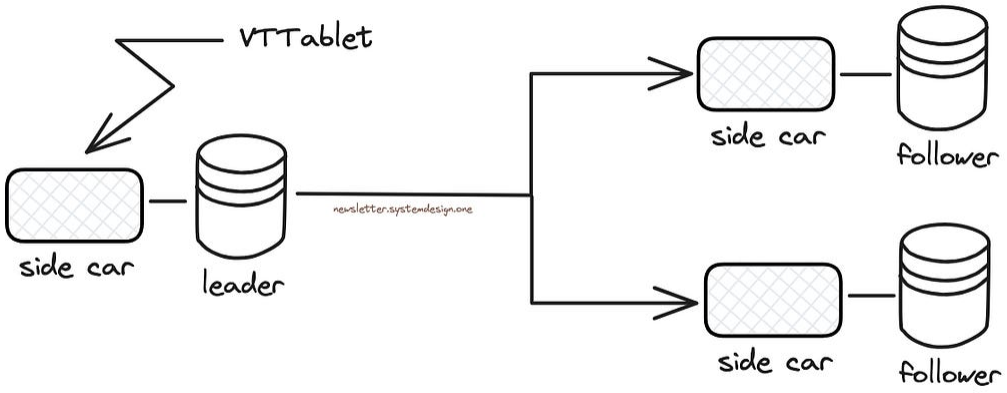
They wanted an abstraction layer on top of MySQL for simplicity and scalability.

So they created Vitess.

Here's how Vitess offers extreme scalability:

1. Interacting with Database:

They installed a sidecar server in front of each MySQL instance and called it **VTTablet**.



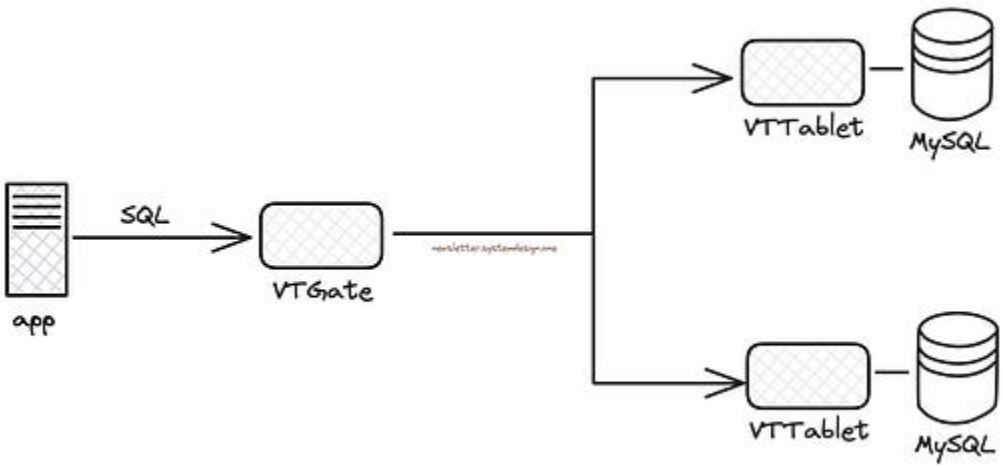
VTTablet Running as a Sidecar Server

It let them:

- Control MySQL server and manage database backups
- Rewrite expensive queries by adding the limit clause
- Cache frequently accessed data to prevent the thundering herd problem

2. Routing SQL Queries:

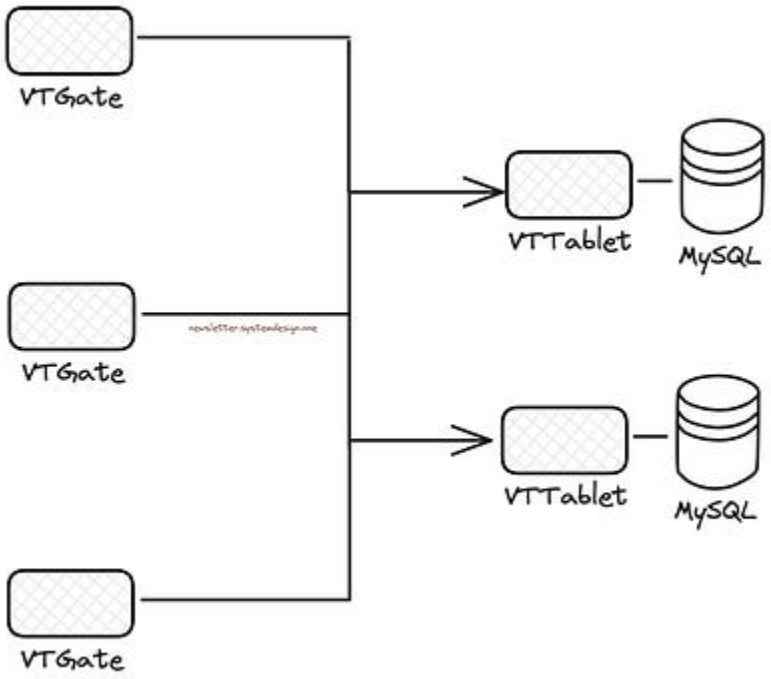
They set up a stateless proxy server to route the queries and called it **VTGate**.



VTGate Routing Queries to the Specific Shard

It let them:

- Find the correct VTTablet to route a query based on the schema and sharding scheme
- Keep the number of MySQL connections low via connection pooling
- Speak MySQL protocol with the application layer
- Act like a monolithic MySQL server for simplicity
- Limit the number of transactions at a time for performance

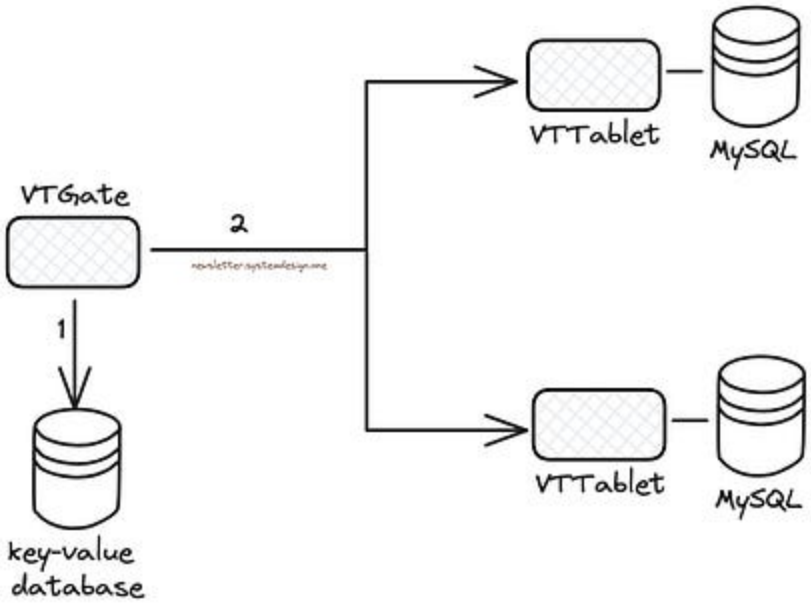


Scaling With Many VTGate Servers

Besides they run many VTGate servers to scale out.

3. State Information:

They set up a distributed **key-value database** to store information about schemas, sharding schemes, and roles.



Key-Value Database Storing Meta Information

Also it takes care of relationships between databases like the leader and followers.

They use Zookeeper to implement the key-value database.

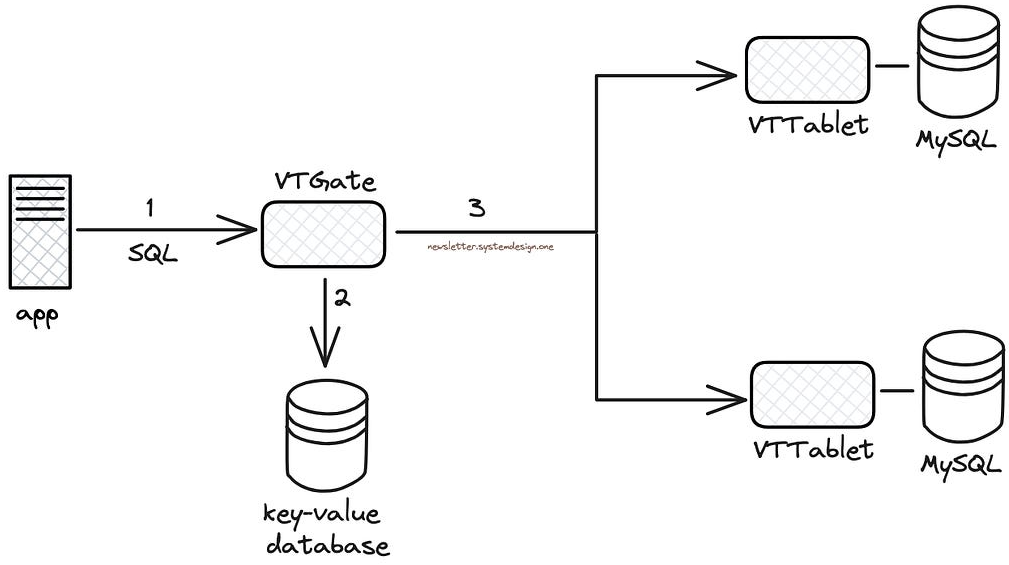
Besides they cache this data on VTGate for better performance.

Updating Key-Value Database

They run an HTTP server to keep the key-value database updated. And called it **VTctld**.

It gets the entire list of servers and their relationships and then updates the key-value database.

TL;DR:



High-Level Architecture of Vitess

- VTGate: proxy server to route queries
- Key-Value Database: configuration server for topology management
- VTTablet: sidecar server running on each MySQL

They wrote Vitess in Go and open-sourced it.

Also it supports MariaDB.

While YouTube was able to serve 2.49 billion users with the Vitess MySQL combination.

This case study shows MySQL can easily handle internet-scale traffic.

Consider subscribing to get simplified case studies delivered straight to your inbox:

Subscribed

Follow me on LinkedIn | YouTube | Threads | Twitter | Instagram

---

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.

1 Referral

Leetcode Master Template

2 Referrals

Interview Mistakes to Avoid PDF

3 Referrals

Popular Interview Questions PDF

Share

How Facebook Scaled Live Video to a Billion Users

NEO KIM • MAY 24, 2024

Read full story

How Razorpay Scaled to Handle Flash Sales at 1500 Requests per Second

NEO KIM • MAY 17, 2024

Read full story

References

- Vitess Official Site
- Vitess Architecture Official Docs

- Scaling YouTube's Backend: The Vitess Trade-offs - @Scale 2014 - Data
- Vitess: A Distributed Scalable Database Architecture
- What Is Vitess?
- Vitess on GitHub
- Scalability at YouTube
- Vitess Supported Databases
- Do Mysql slaves run multiple threads to read the Relay log to sync up the Master's operation
- 11 Reasons Why YouTube Was Able to Support 100 Million Video Views a Day With Only 9 Engineers
- Most Visited Websites In The World (May 2024)